



THE PRAGMATIC ENGINEER • BIG TECH DEEP DIVE

Frontier Engineering

Inside OpenAI's Agentic Software Factory

A deep dive into how Codex and ChatGPT Work have "taken over" internal operations, how the frontier lab builds its agentic software factory, and the engineering challenges of operating at massive scale.

**Gergely Orosz**

Author, The Pragmatic Engineer

Published: September 15, 2026

Location: San Francisco (OpenAI HQ)

CORE FINDINGS & ENGINEERING REALITIES

- **Unlimited Token Budgets:** Engineers, researchers, and non-engineering teams (finance, legal, recruiting) operate with effectively unlimited token budgets, reshaping tool design and enabling intensive agentic workflows.
- **A 9-Step Agentic Software Factory:** OpenAI operates a structured delivery factory where humans steer intent and taste while Codex gathers context, writes code, manages CI, reviews via domain-specialist agents, and monitors deployments.
- **Hockey Stick PR Volumes & 10x CI Surge:** Code generation has accelerated so quickly that CI/CD load increased roughly 10x in just 6 months, creating unprecedented bottlenecks across version control, testing, and deployment.
- **The Native Mobile App Bottleneck:** While web and backend code can ship in minutes, native iOS and Android apps remain constrained by manual App Store review cycles taking days or weeks.
- **Closed-Loop Feedback (Perf Factory & Sevbot):** Production monitoring is no longer just static dashboards; autonomous systems de-duplicate latency regressions, find root causes, and propose automated PR fixes.

1. Codex Takes Over at OpenAI

The takeaway from my visit to the company's headquarters which really sticks out is that Codex – and lately Codex and ChatGPT Work – have taken over *everything* there, starting in around January. Desktop lead, **Andrew Ambrosino**, told me:

"The big theme of the past months has been that everything is now a coding agent. Whether the visible code is your output, or an intermediate step, everything is code. It's writing code to get stuff done."

Andrew Ambrosino — Desktop Lead, OpenAI

Every group at OpenAI uses Codex weekly: from Core Research and Applied Infra, to Legal, Finance, and Recruiting. For non-engineering teams, adoption surged from near 0% to nearly 90% in just four months.

Share of work using Codex at OpenAI, by department

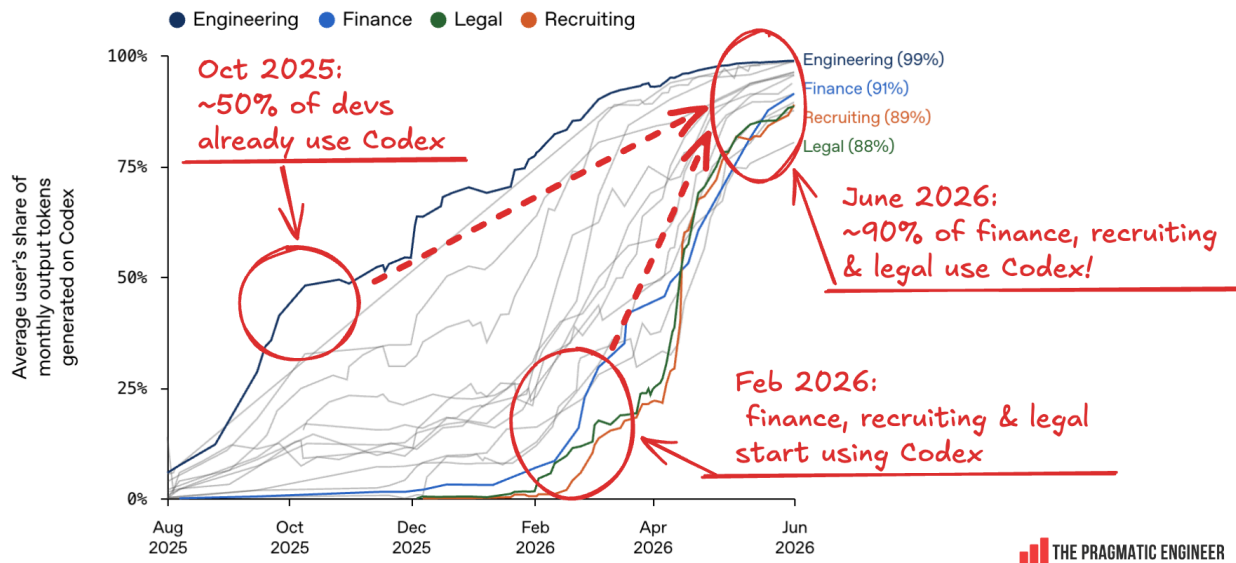


Figure 1: Codex usage since August 2025 at OpenAI by department. Notice the hockey-stick adoption across both engineering and non-engineering departments. Source: OpenAI.

Why did Codex take off so fast internally? Three main factors stood out during my conversations:

- **Unlimited Token Budgets:** Employees don't have to ration tokens or think about per-query costs. This creates an environment where people run long, highly iterative threads with multi-step tools without hesitation.
- **Role-Specific Plugins and Skills:** Teams create and distribute their own skills. Instead of giving developers an empty text box, specialized skills allow the agent to adapt to finance models, legal contracts, or cloud infrastructure setups.
- **Subject Matter Experts Embedded in Engineering:** Because models have become exceptionally capable, domain experts (financial analysts, lawyers, technical writers) are embedded directly within engineering teams to instill "taste" and define quality benchmarks for the agent harnesses.

2. Death of the IDE & Pull Requests

Late last year, the Codex team was torn about whether to release the Codex desktop app. Andrew Ambrosino recalls the hesitation:

"In December 2025, we weren't entirely sure if we would release the Codex app. We had the Codex CLI as a terminal, and there are large, feature-rich IDEs out there... Also, don't forget that in November, Antigravity came out as a VS Code fork. This added to the feeling that perhaps we should have also forked VS Code. But still, we dismissed the temptation and went with our gut feeling that as AI agents get better, IDEs will matter less."

Andrew Ambrosino — Desktop Lead, OpenAI

Indeed, since January, traditional IDE usage has steadily declined at OpenAI. Developers spend most of their time in agent environments, terminal harnesses, or reviewing agent proposals rather than typing in text editors.

The 10x CI/CD Scaling Surge

One direct consequence of agentic productivity is the sheer volume of code flowing through internal developer infrastructure. **Venkat Venkataramani**, VP of Engineering, Applied Infra, explained:

"The number of pull requests (PRs) per engineer is growing like a hockey stick. Every part of the build-test-deploy pipeline is seeing dramatically more load. We're talking about roughly a 10x increase in load on some systems in about six months. At most companies, that kind of growth might happen over two or three years... Every month, we wake up to a new set of infrastructure scaling challenges to solve."

Venkat Venkataramani — VP of Engineering, Applied Infra, OpenAI

In this high-velocity environment, traditional pull requests and manual code reviews are breaking down. When engineers generate dozens of PRs a day, having a single colleague manually inspect every diff is completely unsustainable.

The Native Mobile App Bottleneck

While backend and web deployments can happen rapidly, native iOS and Android apps represent a painful bottleneck. **Sulman Choudhry**, Head of Engineering, ChatGPT, highlighted this growing tension:

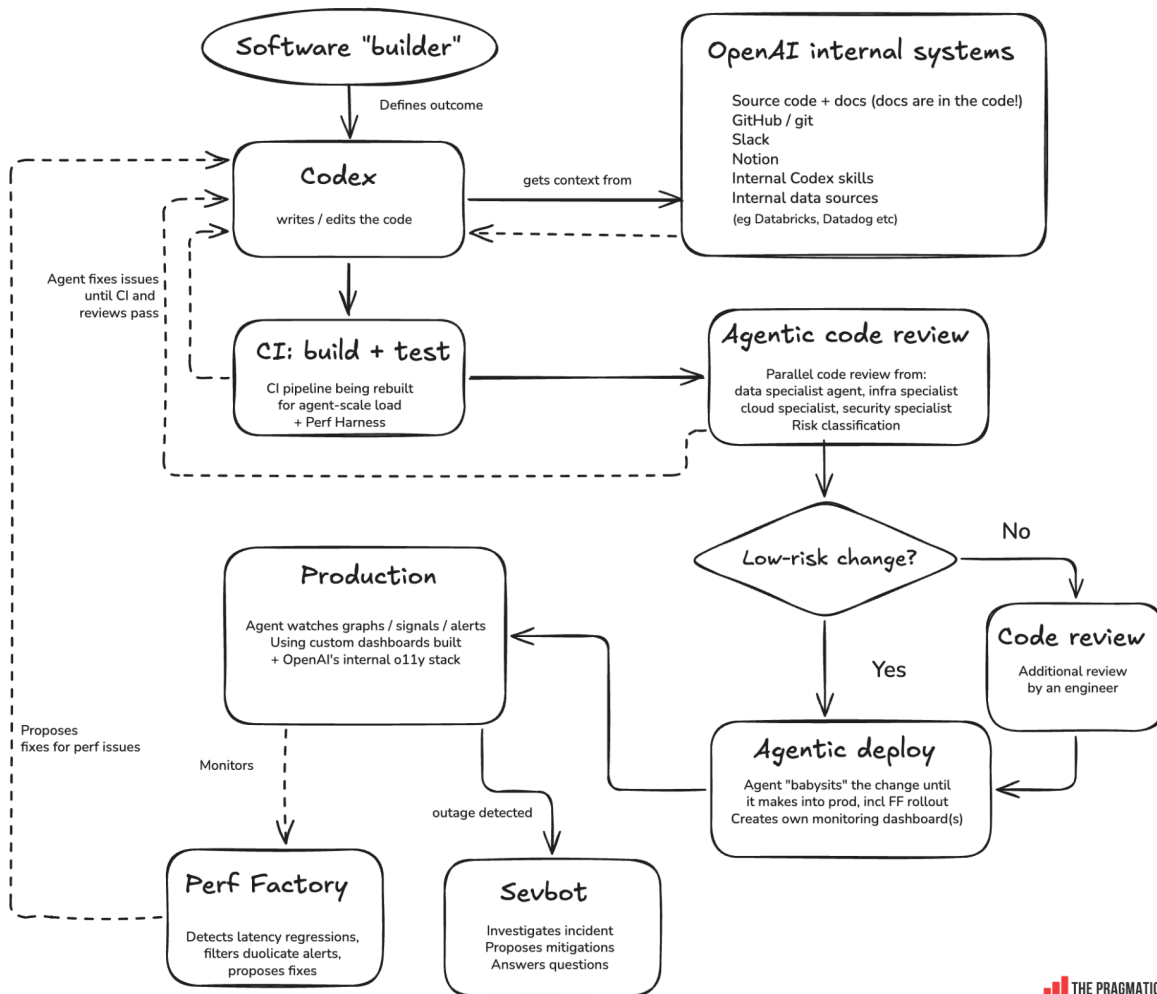
"In the age of Codex, I think we're hitting the next version of this problem. Code generation is getting dramatically faster, but getting that code into users' hands on native mobile is not. For Codex in particular, where usage is heavily mobile-first, that gap is already becoming painful... If software can be written in minutes, waiting days or weeks to get it onto a phone starts to look increasingly absurd."

Sulman Choudhry — Head of Engineering, ChatGPT, OpenAI

3. OpenAI's Agentic Software Factory

The idea of a "software factory" is similar to a physical factory where robots and humans produce vehicles together. In the software context, AI agents and human engineers collaborate systematically along a defined assembly line.

The "agentic software factory" inside OpenAI



THE PRAGMATIC ENGINEER

Figure 2: OpenAI's end-to-end "agentic software factory" pipeline: from human intent to agentic context gathering, build, test, multi-agent review, per-change deployment, and automated feedback loops. Source: OpenAI / The Pragmatic Engineer.

Here is the exact nine-step pipeline operating inside OpenAI Applied Infra:

Step 1 A Human Builder Defines the Desired Outcome

A software engineer or product manager specifies the problem, acceptance criteria, and constraints. Human judgment, prioritization, and "taste" are paramount here. Engineers act increasingly like product managers steering systems rather than manual coders.

Step 2 Codex Gathers Context

OpenAI stores all documentation directly inside the source code repository. Codex accesses Git repos, GitHub issues, Slack channels, Notion specs, Databricks, Datadog metrics, internal logs, and custom Codex skills. New hires are directed to ask Codex questions because it possesses complete cross-repository context.

Step 3 Codex Implements Code Changes

Codex executes iteratively, making code changes across the codebase until it reaches its objective, verifying functionality locally.

Step 4 Build & Test, then CI (Local-CI Gate & Perf Harness)

The agent builds code locally, runs tests, fixes breakages, and opens a PR. The PR triggers CI server linters and test suites. The agent "babysits" the PR until it turns green, fixing any CI failures automatically. Problematic PRs are routed through a *Perf Harness* with Synthetics A/B testing to measure performance regressions.

Step 5 Agentic Code Review (Domain Specialists & Risk Tiers)

Instead of a single generic AI reviewer, OpenAI spins off multiple parallel "domain specialist" agents (cloud infra, data pipelines, application security). Changes are classified by risk: high-risk PRs require multi-agent reviews plus mandatory human sign-off; low-risk changes in approved codebase areas can be auto-approved by agents.

Step 6 Agentic Deploy (Per-Change Autonomous Rollouts)

Once approved, each change is assigned its own dedicated agent with the instruction: *"Handhold this change until it is safely and fully rolled out."* For feature flags, the agent inspects the flag, determines success/failure signals, and dynamically constructs its own monitoring dashboard.

Step 7 Observe Production at Granular Scale

Telemetry includes custom agent-generated dashboards, logs, metrics, traces, and wide events. Instead of static service-level dashboards, observability operates at the granularity of individual code changes.

Step 8 Production Monitoring Feeds Back into Development (Perf Factory)

OpenAI's **Perf Factory** runs agents that sift through alerts and production metrics, de-duplicate noise, identify latency regressions, determine root causes, and automatically propose pull requests to resolve performance regressions.

Step 9 Respond to Outages (Sevbot)

Sevbot is OpenAI's incident-response agent. Upon detecting an anomaly, it activates in Slack, aggregates incident context, and proposes specific mitigation actions. It maintains a strict human-in-the-loop gate: engineers must explicitly authorize mitigations before any production change is executed.

4. How Engineering Tooling & Practices Are Changing

Codex is changing how easy it is to build internal tools and having an impact on standard engineering practices like debugging and system architecture:

- **Ephemeral Internal Tooling:** Instead of dedicating weeks to building internal administrative tools or dashboards, engineers prompt Codex to generate single-use scripts, temporary observability boards, and tailored migration tools on demand.

- **Context-Rich Debugging:** Debugging shifts from reading stack traces in isolation to querying an agent that simultaneously correlates application logs, recent commits, CI run traces, and Slack discussions.
- **The Shift in Engineering Identity:** The role of the engineer is redefined from manual syntax authoring to system architecture, specification design, boundary enforcement, and verification judgment.